



## **Chapter S3C: Structured Query Language**

### **CONTENT**

| <b>No.</b> | <b>Topic</b>                                                                                                                                                                                                                                                                                                                    | <b>Syllabus Guide</b> |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| <b>1</b>   | <b>Introduction to SQL</b>                                                                                                                                                                                                                                                                                                      |                       |
| <b>2</b>   | <b>Using DB Browser for SQLite</b><br><br><b>SQL statements:</b><br><br><b>CREATE TABLE</b><br><br><b>ALTER TABLE</b><br><br><b>DROP TABLE</b><br><br><b>SELECT</b><br><br><b>ORDER BY</b><br><br><b>SQL JOIN</b><br><br><b>UPDATE</b><br><br><b>INSERT INTO</b><br><br><b>DELETE</b><br><br><b>GROUP BY, HAVING, AS, LIMIT</b> | <b>3.3.8</b>          |
| <b>3</b>   | <b>Operators</b>                                                                                                                                                                                                                                                                                                                |                       |
| <b>4</b>   | <b>Functions</b>                                                                                                                                                                                                                                                                                                                |                       |
| <b>5</b>   | <b>Exercises</b>                                                                                                                                                                                                                                                                                                                |                       |
| <b>6</b>   | <b>References</b>                                                                                                                                                                                                                                                                                                               |                       |

## 1. Introduction to SQL

### 1.1 SQL

SQL (Structured Query Language) is the industry-standard language for creating and managing databases. It includes two key parts:

- **DDL (Data Definition Language):** used to create and modify database structures (e.g., CREATE, ALTER, DROP).
- **DML (Data Manipulation Language):** used to query and maintain data (e.g., SELECT, INSERT, UPDATE, DELETE).

SQL scripts can therefore define database tables and manipulate the data stored within them.

### 1.2 SQLite

SQLite is an **open-source, serverless, self-contained relational database engine**. Unlike other systems, it does not require a dedicated database server – all data is stored in a single .db or .sqlite file on disk. Applications such as Python programs interact with this file using a connector (e.g., the sqlite3 module).

The connector:

- Opens and closes connections to the file
- Sends SQL queries
- Retrieves and returns results



Advantages of SQLite

- **Portable:** database stored in one file, easy to move between systems
- **Simple:** minimal setup, no server installation
- **Zero-configuration:** runs directly within an application

### 1.3 SQL and SQLite Data Types

Selecting data types for efficient storage is crucial when creating tables. While **Standard SQL uses a rigid type system**, **SQLite** employs a flexible system called **Type Affinity**. This means a column's declared type is treated as a **recommendation**, or **affinity**, rather than a strict rule. This simplified system maps many standard SQL types to an SQLite equivalent:

| Standard SQL Type | SQLite Equivalent            | Description                                                                    |
|-------------------|------------------------------|--------------------------------------------------------------------------------|
| INT / INTEGER     | INTEGER                      | Whole numbers                                                                  |
| CHAR(n)           | TEXT                         | Fixed-length text (SQLite ignores length)                                      |
| VARCHAR(n)        | TEXT                         | Text string of variable length                                                 |
| DECIMAL / NUMERIC | NUMERIC                      | Decimal or numeric values. SQLite stores as INTEGER or REAL depending on value |
| REAL/FLOAT        | REAL                         | Floating-point numbers                                                         |
| DATE/ DATETIME    | TEXT                         | Date and Time stored as text in 'YYYY-MM-DD HH:MM:SS' format                   |
| BOOLEAN           | NUMERIC<br>(0=False, 1=True) | Boolean values represented as numeric                                          |
| BLOB              | BLOB                         | Binary Large Object. Stores files, images, or raw binary data                  |

## 1.4 SQLite software and tools

We will use **DB Browser for SQLite** v3.10.1 (v3.13.1 as of Oct 25) to create and edit SQLite databases, and Python's **sqlite3** module to access them programmatically.

## 1.5 NoSQL Databases

Traditional SQL databases use fixed tables with predefined schemas and relationships (foreign keys). However, many modern applications, such as social media, chat, or IoT systems, need to store **large or unstructured data** that changes rapidly. In these cases, **NoSQL** ("Not Only SQL") databases are preferred for their flexibility, scalability, and high performance.

Main Types of NoSQL Databases:

| Type        | Example                  | Data Format             | Typical Use                             |
|-------------|--------------------------|-------------------------|-----------------------------------------|
| Document    | <i>MongoDB</i> , CouchDB | JSON-like documents     | Web applications, content management    |
| Key-Value   | Redis                    | Key-value pairs         | Caching, session management             |
| Wide-Column | Cassandra, HBase         | Column families         | Big data analytics, large-scale ops     |
| Graph       | Neo4j                    | Nodes and relationships | Social networks, recommendation engines |

We will use MongoDB in this syllabus.

Example of a MongoDB document in json format:

```
{
  "BookID": 1001,
  "Title": "Life of Pi",
  "Author": "Yann Martel",
  "Damaged": false,
  "Publisher": { "Name": "Penguin", "Country": "UK" }
}
```

Key differences between SQL and NoSQL

| Feature        | SQL                                 | NoSQL                                    |
|----------------|-------------------------------------|------------------------------------------|
| Structure      | Fixed tables with predefined schema | Flexible, schema-less or dynamic schema  |
| Relationships  | Uses joins & foreign keys           | Embedded documents or linked data        |
| Query Language | Structured Query Language (SQL)     | Database-specific (e.g. JSON queries)    |
| Scalability    | Vertical (scale-up: bigger server)  | Horizontal (scale-out: multiple servers) |

## Quiz

1. Which database type is more suitable for large, unstructured social media data?
2. Which database uses fixed tables with predefined schema?
3. Give one advantage of a NoSQL database over a relational one.

## 2. Using DB Browser for SQLite

**DB Browser for SQLite** provides a simple interface to create and manage SQLite databases, run SQL statements and view data.

### 2.1 Creating a Database

SQLite databases are single files with a .sqlite or .db extension. In DB Browser, click **<New Database>**, give the file a name (e.g. library.db), and start adding tables. (Note: the *CREATE DATABASE* command is not used in *SQLite*.)

### 2.2 Entering SQL Statements

Click the **<Execute SQL>** tab to type commands. The following are key toolbar icons:

|                                                                                   |                                                            |
|-----------------------------------------------------------------------------------|------------------------------------------------------------|
|  | Creates a new tab for entering SQL.                        |
|  | Loads SQL file                                             |
|  | Saves SQL entered as a text file (You may save it as .sql) |
|  | Execute all SQL statements in that tab.                    |
|  | Execute the current line only.                             |

After executing SQL statement or making changes, you can click the following:

|                                                                                                    |                                                           |
|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
|  Write Changes    | Save changes made into the file.                          |
|  Revert Changes | Undo changes made to the database, i.e. reload from file. |

We will create a `library` database containing `Book`, `Publisher`, `Borrower` and `Loan` tables.

### CREATE TABLE

Use the `CREATE TABLE` statement to define tables, primary keys and foreign keys. Each field should have a suitable data type based on standard SQL and SQLite equivalents:

```
CREATE TABLE table_name (
    column1_name COLUMN1_TYPE COLUMN1_CONSTRAINTS,
    column2_name COLUMN2_TYPE COLUMN2_CONSTRAINTS,
    .....
    PRIMARY KEY (column1_name, column2_name, .....),
    FOREIGN KEY (column_name) REFERENCES table_name(column_name)
);
```

For example, to create the `Borrower` table:

```
CREATE TABLE Borrower (
    BorrowerID INTEGER,
    FirstName TEXT,
    Surname TEXT,
    ContactNumber INTEGER,
    PRIMARY KEY (BorrowerID)
);
```

Next, create the Book table:

```
CREATE TABLE Book (  
  BookID INTEGER,  
  Title TEXT,  
  PublisherID INTEGER,  
  Damaged INTEGER,  
  FOREIGN KEY(PublisherID) REFERENCES Publisher(PublisherID),  
  PRIMARY KEY(BookID)  
);
```

Damaged uses INTEGER to represent Boolean values (0 = false, 1 = true).

SQL is not case-sensitive, and statements can span multiple lines.

Indentation is not significant for SQL code. SQL code can be written in a single line or spread across multiple lines and separated using a semi-colon (;).

Next create the Publisher and Loan tables with their respective fields:

```
CREATE TABLE Publisher (PublisherID INTEGER, Name TEXT,  
  Primary Key (PublisherID) );  
  
CREATE TABLE Loan (LoanID INTEGER, BorrowerID INTEGER,  
  BookID INTEGER, "Date Borrowed" TEXT,  
  PRIMARY KEY(LoanID),  
  FOREIGN KEY(BookID) REFERENCES Book(BookID),  
  FOREIGN KEY(BorrowerID) REFERENCES Borrower(BorrowerID) );
```

After creating all four tables, view their structure under the <Database Structure> tab.

## ALTER TABLE

The ALTER TABLE is used to add, delete or modify fields and constraints in an existing table. For example, to add an Email field to the Borrower table:

- ALTER TABLE Borrower ADD Email TEXT;

To rename the Borrower table to Member table:

- ALTER TABLE Borrower RENAME TO Member;

## DROP TABLE

The DROP TABLE deletes the entire table and all its data. Once deleted, no further entries can be added to that table. For example, to remove the Loan table:

- DROP TABLE Loan;

At this point, you should know how to create a database, create and modify tables, and delete tables. Close the database via File>Close Database.

## 2.3 Load database

Open the library database in DB Browser for SQLite via `<Open Database>`. Explore the `<Database Structure>` and `<Browse Data>` tabs.

Using Data Manipulation Language (DML) statements (`SELECT`, `INSERT`, `UPDATE` and `DELETE`), you can easily retrieve, edit or delete data.

### SELECT

The `SELECT` statement allows the user to retrieve data from the tables.

Select all fields:

- `SELECT * FROM Book;`

Retrieve all the damaged books.

- `SELECT * FROM Book WHERE Damaged = 1;`

Retrieve books with no publishers using the `IS` operator.

- `SELECT * FROM Book WHERE PublisherID IS NULL;`

Retrieve all books where there is a publisher.

- `SELECT * FROM Book WHERE PublisherID IS NOT NULL;`

Retrieves all books with Title 'Life of Pi' and are not damaged.

- `SELECT * FROM Book WHERE Title = 'Life of Pi' AND Damaged = 0;`

Retrieve the titles of the books in the `Book` table.

- `SELECT Title FROM Book;`

### ORDER BY

Listing all the loans (default ascending) ordered by ID:

- `SELECT * FROM LOAN;`

| ID | BorrowerID | BookID | Date Borrowed |
|----|------------|--------|---------------|
| 1  | 3          | 2      | 20180220      |
| 2  | 3          | 1      | 20171215      |
| 3  | 2          | 3      | 20171231      |
| 4  | 1          | 5      | 20180111      |

Ascending order by `BookID`.

- `SELECT * FROM Loan ORDER BY BookID ASC;`

| ID | BorrowerID | BookID | Date Borrowed |
|----|------------|--------|---------------|
| 2  | 3          | 1      | 20171215      |
| 1  | 3          | 2      | 20180220      |
| 3  | 2          | 3      | 20171231      |
| 4  | 1          | 5      | 20180111      |

Descending order by `BookID`.

- `SELECT * FROM Loan ORDER BY BookID DESC;`

| ID | BorrowerID | BookID | Date Borrowed |
|----|------------|--------|---------------|
| 4  | 1          | 5      | 20180111      |
| 3  | 2          | 3      | 20171231      |
| 1  | 3          | 2      | 20180220      |
| 2  | 3          | 1      | 20171215      |

Count total records:

- `SELECT COUNT(*) FROM Loan;`

If `ASC` or `DESC` is not specified, results are sorted in ascending order by default.

## SQL JOIN

A `JOIN` combines data from two tables.

Common types of join: **cross join**, **inner join** and **left outer join**.

**Cross join** returns **all combinations** of rows from both tables.

- `SELECT * FROM Book, Publisher;`

| ID  | Title                        | PublisherID | Damaged | ID | Name         |
|-----|------------------------------|-------------|---------|----|--------------|
| 1   | The Lone Gatsby              | 5           | 0       | 1  | NPH          |
| 1   | The Lone Gatsby              | 5           | 0       | 2  | Unpop        |
| 1   | The Lone Gatsby              | 5           | 0       | 3  | Appleson     |
| 1   | The Lone Gatsby              | 5           | 0       | 4  | Squirrel     |
| 1   | The Lone Gatsby              | 5           | 0       | 5  | Yellow Flame |
| 2   | A Winter's Slumber           | 4           | 1       | 1  | NPH          |
| 2   | A Winter's Slumber           | 4           | 1       | 2  | Unpop        |
| 2   | A Winter's Slumber           | 4           | 1       | 3  | Appleson     |
| 2   | A Winter's Slumber           | 4           | 1       | 4  | Squirrel     |
| 2   | A Winter's Slumber           | 4           | 1       | 5  | Yellow Flame |
| 3   | Life of Pi                   | 4           | 0       | 1  | NPH          |
| 3   | Life of Pi                   | 4           | 0       | 2  | Unpop        |
| 3   | Life of Pi                   | 4           | 0       | 3  | Appleson     |
| 3   | Life of Pi                   | 4           | 0       | 4  | Squirrel     |
| 3   | Life of Pi                   | 4           | 0       | 5  | Yellow Flame |
| 4   | A Brief History Of Primates  | 3           | 0       | 1  | NPH          |
| 4   | A Brief History Of Primates  | 3           | 0       | 2  | Unpop        |
| 4   | A Brief History Of Primates  | 3           | 0       | 3  | Appleson     |
| 4   | A Brief History Of Primates  | 3           | 0       | 4  | Squirrel     |
| 4   | A Brief History Of Primates  | 3           | 0       | 5  | Yellow Flame |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 1  | NPH          |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 2  | Unpop        |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 3  | Appleson     |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 4  | Squirrel     |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 5  | Yellow Flame |
| 6   | The Catcher in the Eye       | 1           | 1       | 1  | NPH          |
| 6   | The Catcher in the Eye       | 1           | 1       | 2  | Unpop        |
| 6   | The Catcher in the Eye       | 1           | 1       | 3  | Appleson     |
| 6   | The Catcher in the Eye       | 1           | 1       | 4  | Squirrel     |
| 6   | The Catcher in the Eye       | 1           | 1       | 5  | Yellow Flame |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | 1  | NPH          |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | 2  | Unpop        |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | 3  | Appleson     |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | 4  | Squirrel     |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | 5  | Yellow Flame |

This produces a Cartesian product; all `Book` rows paired with all `Publisher` rows.

Notice that there are two columns of `ID`. The first `ID` column is from `Book` table, while the second `ID` column is from the `Publisher` table.

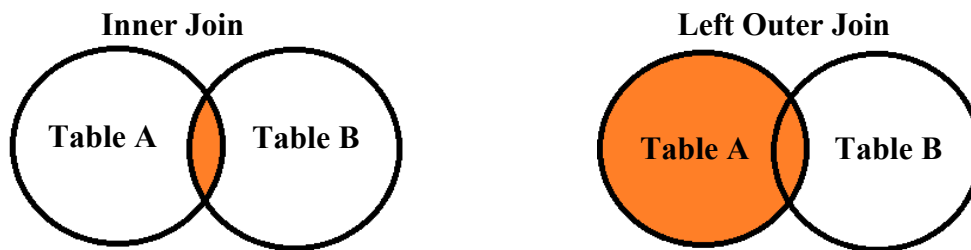
The table above is big, with a lot of necessary rows. It is more useful to have the publisher of the title of each book. Add a condition to do that.

To show only books that have a matching publisher, use an **inner join**.

- `SELECT * FROM Book INNER JOIN Publisher ON Book.PublisherID = Publisher.ID;`

| ID | Title                       | PublisherID | Damaged | ID | Name         |
|----|-----------------------------|-------------|---------|----|--------------|
| 1  | The Lone Gatsby             | 5           | 0       | 5  | Yellow Flame |
| 2  | A Winter's Slumber          | 4           | 1       | 4  | Squirrel     |
| 3  | Life of Pi                  | 4           | 0       | 4  | Squirrel     |
| 4  | A Brief History Of Primates | 3           | 0       | 3  | Appleson     |
| 5  | To Praise a Mocking Bird    | 2           | 0       | 2  | Unpop        |
| 6  | The Catcher in the Eye      | 1           | 1       | 1  | NPH          |

This table is more meaningful as it returns only books with a publisher. Notice that the title 'H2 Computing Ten Year Series' is not included as it has no `PublisherID`.



**Inner join** selects all records from Table A and Table B which meet the join condition.

**Left outer join** selects all records from Table A, along with records from Table B which meet the join condition.

If the **inner join** is changed to **left outer join**:

- `SELECT * FROM Book LEFT OUTER JOIN Publisher ON Book.PublisherID = Publisher.ID`

All records in `Book` are included in the **left outer join**, even books who have no publishers.

| ID  | Title                        | PublisherID | Damaged | ID   | Name         |
|-----|------------------------------|-------------|---------|------|--------------|
| 1   | The Lone Gatsby              | 5           | 0       | 5    | Yellow Flame |
| 2   | A Winter's Slumber           | 4           | 1       | 4    | Squirrel     |
| 3   | Life of Pi                   | 4           | 0       | 4    | Squirrel     |
| 4   | A Brief History Of Primates  | 3           | 0       | 3    | Appleson     |
| 5   | To Praise a Mocking Bird     | 2           | 0       | 2    | Unpop        |
| 6   | The Catcher in the Eye       | 1           | 1       | 1    | NPH          |
| 123 | H2 Computing Ten Year Series | NULL        | 0       | NULL | NULL         |

Notice there are two columns of ID. Use table aliases or rename columns to avoid confusion when both tables have similar column names (e.g., ID).

**Quiz**

4. Which of the following SQL statements show all data inside the Publisher table? (Tick the valid statements)

- ☐ Select all FROM Publisher  
☐ Select \* FROM Publisher  
☐ Select ID, Name FROM Publisher  
☐ Select ID, Name, PublisherID, Damaged FROM Publisher

5. Write down the SQL statement to show all the Names of the publishers inside the Publisher table.

6. What does the following SQL statement do?

```
SELECT Title FROM Book WHERE PublisherID=1 AND Damaged=0
```

7. Write down the SQL statement to show all the Titles of the books which have PublisherID=1 or 2.

8. Write down the SQL statement to select all the records in Loan table arranged in ascending order of BookID with BorrowerID = 1.

9. Write down the SQL statement to join all the books that are not damaged with their Publishers.

## UPDATE

The `UPDATE` command modifies existing data. Multiple records can be updated at once.

Prefix all the book titles with “Book”

- `UPDATE Book Set Title = 'Book: ' || Title;`

Each book title now starts with ‘Book:’. The use of `||` is for string concatenation.

Update the `Loan` table because all the books borrowed by Kumara (`BorrowerID=3`) should have been borrowed by Sarah (`BorrowerID=2`).

- `UPDATE Loan Set BorrowerID = 2 WHERE BorrowerID = 3;`

## INSERT INTO

The `INSERT INTO` command adds new records to a table.

To insert a new publisher:

- `INSERT INTO Publisher(Name) VALUES ('BigBooks')`

This will add the publisher "BigBooks" into the `Publisher` table.

Verify by viewing the `Publisher` table under **<Browse Data>**.

## DELETE

The `DELETE` command removes records from a table.

This deletes the book 'Life of Pi' from the `Book` table. Text is case-sensitive.

- `DELETE FROM Book WHERE Title = 'Life of Pi';`

This will delete all entries from the `Loans` table! But the table itself remains.

- `DELETE FROM Loan;`

To remove the table entirely, use `DROP TABLE Loan`.

## GROUP BY, HAVING, AS, LIMIT

The `GROUP BY` statement groups rows that have the same values into summary rows. It is often used with aggregate functions (`COUNT()`, `MAX()`, `MIN()`, `SUM()`, `AVG()`).

The `HAVING` clause filters groups **after aggregation**.

`AS` renames columns or aggregated results for readability.

`LIMIT` restricts the number of results returned.

Count books by publisher.

- ```
SELECT COUNT(BookID), PublisherID
FROM Book
GROUP BY PublisherID;
```

Count books by publisher in descending order.

- ```
SELECT COUNT(BookID), PublisherID
FROM Book
GROUP BY PublisherID
ORDER BY COUNT(PublisherID) DESC;
```

Count books by publisher only if more than 2

- ```
SELECT PublisherID, COUNT(BookID)
FROM Book
GROUP BY PublisherID
HAVING COUNT(BookID) > 2;
```

Find the borrower who borrowed the most books.

- ```
SELECT Borrower.BorrowerID, Borrower.FirstName,
COUNT(Loan.BookID) AS BooksBorrowed
FROM Borrower
INNER JOIN Loan ON Borrower.BorrowerID = Loan.BorrowerID
GROUP BY Borrower.BorrowerID, Borrower.FirstName
ORDER BY BooksBorrowed DESC
LIMIT 1;
```

Top 5 borrowers with more than 2 books, sorted from the most books borrowed to the least.

- ```
SELECT BorrowerID, COUNT(BookID) AS TotalBooks
FROM Loan
GROUP BY BorrowerID
HAVING COUNT(BookID) > 2
ORDER BY TotalBooks DESC
LIMIT 5;
```

### Quiz

10. All the books borrowed by Sarah (`BorrowerID=2`) should have been borrowed by Kumara (`BorrowerID=3`). Write down the SQL statement to update the `Loan` table.

11. Which of the following SQL statements are valid? (Tick the valid statements)

- ☐ `DELETE FROM Book WHERE Title = 'H2 Computing Ten Year Series'`  
☐ `DELETE FROM Book WHERE Title = *`  
☐ `DELETE FROM Book WHERE`  
☐ `DELETE FROM Book`

12. Create a table `Testing` with fields `name`, `tag_no` and `remarks`. The fields `name` and `remarks` should accept text, while `tag_no` is an integer automatically incremented.

The primary key is `tag_no`, `name` field cannot be `NULL`.

```
CREATE TABLE Testing (  
    name TEXT NOT NULL,  
    tag_no INTEGER PRIMARY KEY AUTOINCREMENT,  
    remarks TEXT  
);
```

**Tip:** You can always click <Revert Changes> to remove the recent changes made.

13. Try keying `DROP TABLE Publisher` to remove the table containing the publishers. Is it possible? Explain your answer.

14. Write an SQL statement to list all books along with their publishers (if any), sorted alphabetically by book title.

### 3. Operators

Operators are commonly used in `SELECT`, `UPDATE` and `DELETE`. They are grouped into comparison, logical, string and arithmetic operators.

#### 3.1 Comparison Operators

| Operator              | Description                                                       |
|-----------------------|-------------------------------------------------------------------|
| <code>=</code>        | Checks if two values are equal.                                   |
| <code>!=</code>       | Checks if two values are not equal.                               |
| <code>&lt;&gt;</code> | Checks if two values are not equal.                               |
| <code>&gt;</code>     | Checks if left operand is greater than right operand.             |
| <code>&lt;</code>     | Checks if left operand is less than right operand.                |
| <code>&gt;=</code>    | Checks if left operand is greater than or equal to right operand. |
| <code>&lt;=</code>    | Checks if left operand is less than or equal to right operand.    |

Example: Retrieves all student records where the age is 20 or older.

- `SELECT * FROM Student WHERE Age >= 20;`

#### 3.2 Logical Operators

| Operator      | Description                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------|
| <b>AND</b>    | Combines multiple conditions in a <code>WHERE</code> clause; all conditions must be true.         |
| <b>OR</b>     | Combines multiple conditions in a <code>WHERE</code> clause; at least one condition must be true. |
| <b>IS</b>     | Checks if a value exists (e.g.: <code>IS NULL</code> ).                                           |
| <b>IS NOT</b> | Checks if a value does not exist. (e.g.: <code>IS NOT NULL</code> ).                              |

Example: Retrieves all students who are aged 20 or older and enrolled in course CS101.

- `SELECT * FROM Student  
WHERE Age >= 20 AND CourseCode = 'CS101';`

#### 3.3 LIKE & IN Operators

| Operator    | Description                                                                                                      |
|-------------|------------------------------------------------------------------------------------------------------------------|
| <b>LIKE</b> | Used for pattern matching with <code>%</code> (any sequence of characters) or <code>_</code> (single character). |
| <b>IN</b>   | Checks if a value matches any value in a list.                                                                   |

Examples:

-- Students whose first name starts with D or S

- `SELECT * FROM Student  
WHERE FirstName LIKE 'D%' OR FirstName LIKE 'S%';`

-- Students enrolled in CS101 or CS102

- `SELECT * FROM Student  
WHERE CourseCode IN ('CS101', 'CS102');`

-- Students whose first name starts with D or S and enrolled in CS101 or CS102

- `SELECT * FROM Student  
WHERE (FirstName LIKE 'D%' OR FirstName LIKE 'S%')  
AND CourseCode IN ('CS101', 'CS102');`

-- Retrieve students whose first names have exactly 4 letters and start and end with 'A'

- `SELECT * FROM Student WHERE FirstName LIKE 'A__A';`

### 3.4 Arithmetic Operators

| Operator | Description         |
|----------|---------------------|
| +        | Addition            |
| -        | Subtraction         |
| *        | Multiplication      |
| /        | Division            |
| %        | Modulus (remainder) |

Example:

-- Retrieve each student's original mark and the same mark increased by 10%.

- `SELECT Marks, Marks * 1.1 AS AdjustedMarks FROM Student;`

### 3.5 String Concatenation Operator

| Operator | Description           |
|----------|-----------------------|
|          | Concatenates strings. |

Example:

-- Combines the first and last names of each student into a single full name

- `SELECT FirstName || ' ' || LastName AS FullName FROM Student;`

## 4. Functions

Aggregate functions summarise data in columns.

| Function     | Description                               | Example                                                                 |
|--------------|-------------------------------------------|-------------------------------------------------------------------------|
| <b>MIN</b>   | Returns the minimum value in a column     | <code>SELECT MIN(Marks) AS LowestMark<br/>FROM Student;</code>          |
| <b>MAX</b>   | Returns the maximum value in a column     | <code>SELECT MAX(Marks) AS HighestMark<br/>FROM Student;</code>         |
| <b>SUM</b>   | Returns the sum of all values in a column | <code>SELECT SUM(Marks) AS TotalMarks<br/>FROM Student;</code>          |
| <b>COUNT</b> | Returns the number of values in a column  | <code>SELECT COUNT(StudentID) AS<br/>TotalStudents FROM Student;</code> |
| <b>AVG</b>   | Returns the average value of a column     | <code>SELECT AVG(Marks) AS AverageMarks<br/>FROM Student;</code>        |

## 5. Exercises

### 5.1 Temasek Polytechnic data

Open the file `poly.db`. The database provides the enrolment in different polytechnic courses in Temasek Polytechnic (TP) by year and gender.

**Write SQL commands to retrieve the following data:**

1. List the courses in TP that have more than 250 males in 2022.
2. List the courses in TP that have no more than 50 females in 2021.
3. List out all fields in the table for courses not in School of Informatics & IT in 2020.
4. List the largest number of female(s) in a course in 2022.
5. List the smallest number of male(s) in a course in 2022.
6. Rename all the diplomas called 'Diploma in Cyber & Digital Security' to 'Diploma in Cyber and Digital Security'.
7. Update the table to add 5 students to each gender and each course in School of Design in 2021.
8. List the total number of male and female students enrolled by year. Output in order of year then gender.
9. List the total number of students enrolled in each course by year.
10. List by year, the course(s) which has an enrolment of less than 10.
11. List all the courses offered by the polytechnic for the 3 years.
12. List all the courses offered by the polytechnic, together with its enrolment in 2022.
13. List the highest enrolment of each course over the 3 years, with the year indicated.
14. List the 10 courses with the highest enrolment in 2022, from the highest to the lowest.
15. List courses in 2022 that have more than 100 students.

### 5.2 University of Technology data

- Create a new database `Uni_Technology` in your folder.
- Three csv files are provided – `COURSE.csv`, `STUDENT.csv`, `TUTOR.csv`. The first line contains field names of the table.
- Import all the data from each of the tables into the `Uni_Technology` database. Name the tables according to the filenames – `Course`, `Student` and `Tutor`.
- For `Student` table, set `Student_ID` as primary key, `Course_Code` as foreign key linking to `Course` table, `Tutor_Code` as foreign key linking to `Tutor` table.
- For `Course` table, set `Code` as primary key, and change data type of `Full_Time` to `INTEGER` (boolean).
- For `Tutor` table, set `Code` as primary key, and change data type of `Salary` to `REAL`, and `Contract` to `NUMERIC`.
- Export the database schema to an SQL (text) file.

**Write SQL commands to retrieve the following data:**

1. List of first name and last name from `STUDENT` table.
2. List of first name, last name and zip code from `STUDENT` table in ascending order of zip code, followed by descending order of first name.
3. List of first name, last name, course code, tutor code = 'CTY' from `STUDENT` table, in ascending order of course code.

4. List of first name, last name, course code, tutor code from STUDENT table, in ascending order of first name, where the first name starts from 'D' to 'Z'.
5. List of first name, last name, course code, tutor code from STUDENT table, in ascending order of first name, where the first name contains the letters 'at'.
6. List of first name, last name, course code, tutor code and zip code from STUDENT table, in ascending order of zip code, where zip code has values of 4305 or 4306.
7. List of student ID, first name and last name of student, tutor code of student, and tutor's first name and last name of student.
8. List of all students with their tutor and course of study, in order of student ID.
9. List of courses and the students assigned to each course using these fields: Course\_Code, Course\_Title, Student\_ID, First\_Name and Last\_Name.
10. Find the course with the most students. Output course code, course title and number of students from that course.
11. Change the value of student's address which contains 'Central' to 'Centre'.
12. Insert a new course record with course code = 'CO-SE-1', course title = 'Cybersecurity', Faculty = 'Computing', Level = 'BSc', and full time is True.
13. Delete the course record with course code = 'CO-SE-1'.
14. Find the total number of students with zip code = 4303.
15. Find the highest, lowest, total and average salary (to 2 decimal places) of all tutors.
16. List the tutors whose salaries are above the average.
17. Find the courses that have more than 6 students.
18. List courses that have more than 5 students, grouped by Course\_Code.

### 5.3 Using Python with SQL

Python can access and manipulate SQLite databases using the built-in `sqlite3` module.

```
import sqlite3

# Connect to database (creates if not exist)
connection = sqlite3.connect("library.db")
cursor = connection.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Publisher (
    PublisherID INTEGER PRIMARY KEY,
    Name TEXT) """)

# Insert data
cursor.execute("INSERT INTO Publisher (Name) VALUES (?)",
("BigBooks",))

# Retrieve and display data
cursor.execute("SELECT * FROM Publisher")
for row in cursor.fetchall():
    print(row)

# Save changes and close connection
connection.commit()
connection.close()
```

This example:

- connects Python to an SQLite database file,
- executes SQL statements directly from Python,
- retrieves and displays data from the table.

## 5.4 Using Python with a NoSQL Database (MongoDB)

Python can access MongoDB using the `pymongo` library.

```
from pymongo import MongoClient

# Connect to MongoDB server
client = MongoClient("127.0.0.1", 27017)

# Select database and collection
db = client["library"]
collection = db["books"]

# Insert document
book = {"Title": "Life of Pi", "Author": "Yann Martel",
        "Damaged": False}
collection.insert_one(book)

# Query documents
for doc in collection.find({"Damaged": False}):
    print(doc)
```

This example:

- Connects Python to a MongoDB server.
- Inserts documents into a collection.
- Retrieves and displays documents based on a query.

## 6. References

- <https://sqlitebrowser.org/>
- <https://www.sqlite.org/>
- <https://www.sqlitetutorial.net/what-is-sqlite/>
- <https://www.bbc.co.uk/bitesize/guides/zmh8hbk/revision/1>
- <https://www.w3schools.com/sql/>
- <https://www.tutorialspoint.com/sql/sql-and-or-clauses.htm>